마이크로피펫 OCR인식 및 분주용량 조절

폴더 구조

- gui/ : PyQt5 관련 GUI 관련 코드
  - panels/
    - pippette_panel.py
    - run_status_panel.py
    - target_panel.py
    - run_status_panel.py
    - target_panel.py
    - video_panel.py
    - yolo_panel.py
  - **init**.py
  - controller.py
  - main_window.py
  - main.py
- models/ : ocr 및 yolo 의 Pytorch 및 TensorRT 코드가 담긴
  - ocr/
  - yolo/
- snapshots/ : 테스트 진행 당시 저장을 담당했던 폴더.
- test/ : 간단히 테스트할 때 사용하는 폴더.
  - single_target_paddleOCR_test.py
  - test_paddleOCR.py
  - test_utils.py
- state/ : 카메라 프레임 캡쳐, roi 인식 등의 정보가 일시적으로 저장되는 장소
  - last_frame.jpg
  - last_yolo.jpg
  - rois.json
- worker/ : 핵심 로직들을 담아두는 폴더
  - actuator_linear.py

    - actuator_volumne_dc.py
    - camera.py
    - capture_frame.py
    - control_worker.py
    - make_packet.py
    - motor_controller.py
    - ocr_paddle.py
    - ocr_trt.py
    - paths.py
    - serial_controller.py
    - test_ocr_only.py
    - worker_paddle.py
    - worker.py
    - yolo_worker.py
- calibration_paddle.json : 캘리브레이션된 값이 저장되는 파일

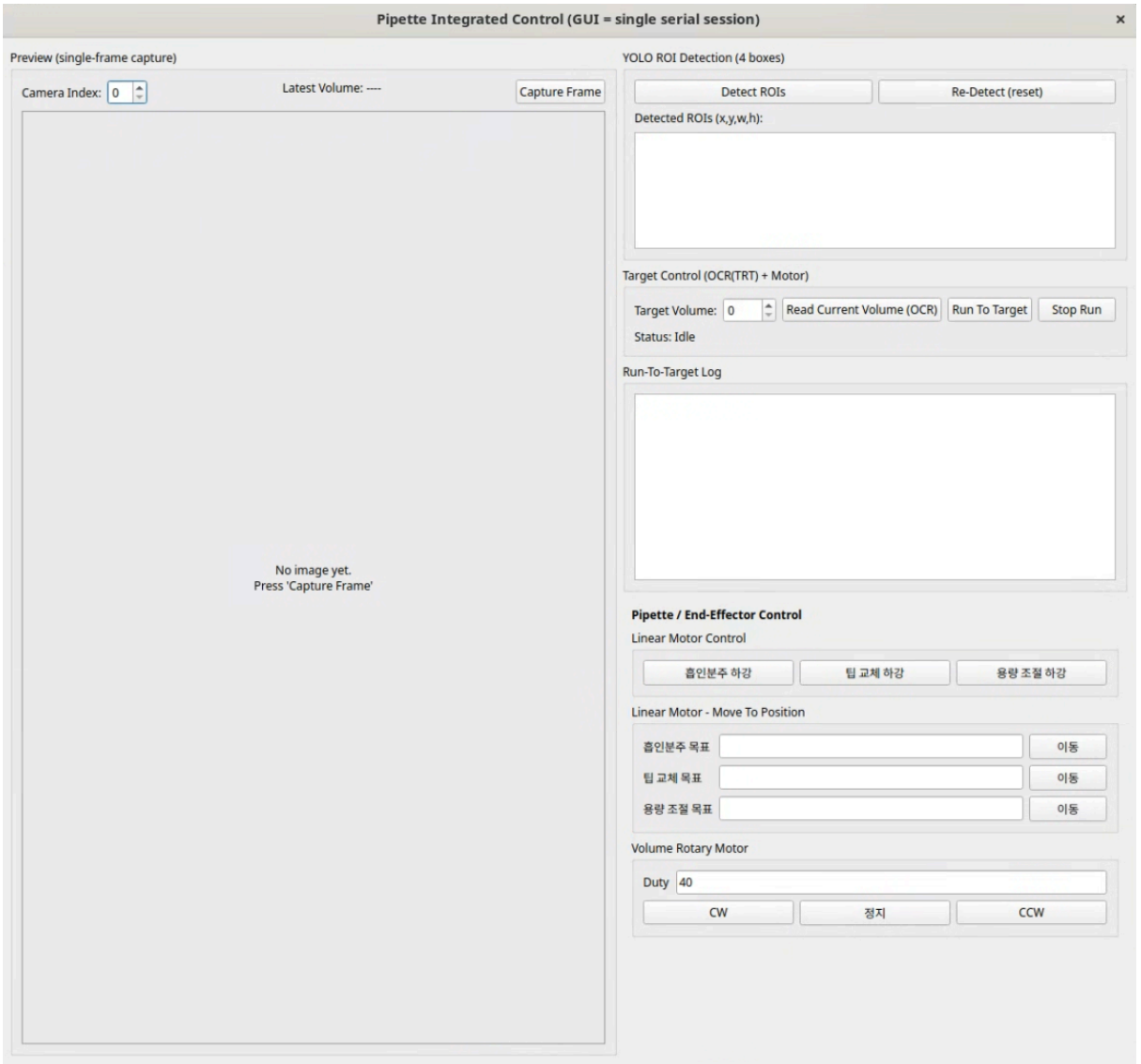
기본 설명

해당 프로젝트는 기본적으로 시스템 파이썬(3.12)를 사용하며, OCR과 같은 이미지 처리에는 현 시점에서 가장 안정적인 3.10을 기준으로 python을 사용합니다.

메인 GUI를 띄우는 폴더(/gui) 는 시스템 파이썬이며, controller에서 worker/에 있는 로직 파일들을 필요에 따라 3.10 환경의 conda로 호출하는 방식으로 사용합니다.

OCR

현재 OCR 모델은 PaddleOCR(오픈소스 모델) + Custom OCR(실제 환경 데이터로 학습시킨 EfficientNet-B0 기반 모델)의 복합 구조로 이루어져 있습니다.
현재 실험환경 특성상 마이크로피펫의 분주량 눈금 부분에 빛번짐 및 노이즈가 쉽게 생기는 환경이라 혹시 인식률이 저조하다면 해당 환경에 대한 차이가 문제일듯 싶습니다.
(노이즈가 포함된 데이터를 가지고 일단 최대한 학습시켜둔 상태입니다.)



기타 사항

또한 GUI 화면에서 'Run to Target'은 아직 연결되어 있지 않습니다.
test/single_target_paddleOCR_test.py를 참고하여 연결하시면 될 거 같습니다.

또한 현재 ‘Read Current Volume(OCR)’버튼 클릭 시 CustomOCR만으로 OCR을 수행하도록 되어있습니다. 추후에 GUI를 수정할 때 test/test_paddleOCR.py를 보고 수정해주시면 될 거 같습니다.